

Absent – after row 5

Language Modeling

Generalization and zeros





The Shannon Visualization Method

- Choose a random bigram
($\langle s \rangle$, w) according to its probability
- Now choose a random bigram
(w , x) according to its probability
- And so on until we choose $\langle /s \rangle$
- Then string the words together

```

<s> I
    I want
      want to
        to eat
          eat Chinese
            Chinese food
              food </s>

I want to eat Chinese food
    
```



Approximating Shakespeare

Unigram

To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have
 Every enter now severally so, let
 Hill he late speaks; or! a more to leg less first you enter
 Are where exeunt and sighs have rise excellency took of.. Sleep knave we. near; vile like

Bigram

What means, sir. I confess she? then all sorts, he is trim, captain.
 Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.
 What we, hath got so she that I rest and sent to scold and nature bankrupt, nor the first gentleman?

Trigram

Sweet prince, Falstaff shall die. Harry of Monmouth's grave.
 This shall forbid it should be branded, if renown made it empty.
 Indeed the duke; and had a very good friend.
 Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

Quadrigram

King Henry.What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;
 Will you not tell me who I am?
 It cannot be but so.
 Indeed the short and the long. Marry, 'tis a noble Lepidus.



Shakespeare as corpus

- $N=884,647$ tokens, $V=29,066$
- Shakespeare produced 300,000 bigram types out of $V^2= 844$ million possible bigrams.
 - So 99.96% of the possible bigrams were never seen (have zero entries in the table)
- Quadrigrams worse: What's coming out looks like Shakespeare because it *is* Shakespeare



The wall street journal is not shakespeare (no offense)

Unigram

Months the my and issue of year foreign new exchange's september were recession ex-
change new endorsed a acquire to six executives

Bigram

Last December through the way to preserve the Hudson corporation N. B. E. C. Taylor
would seem to complete the major central planners one point five percent of U. S. E. has
already old M. X. corporation of living on information such as more frequently fishing to
keep her

Trigram

They also point to ninety nine point six billion dollars from two hundred four oh six three
percent of the rates of interest stores as Mexico and Brazil on market conditions



The perils of overfitting

- N-grams only work well for word prediction if the test corpus looks like the training corpus
 - In real life, it often doesn't
 - We need to train robust models that generalize!
 - One kind of generalization: Zeros!
 - Things that don't ever occur in the training set
 - But occur in the test set



Zeros

- Training set:
 - ... denied the allegations
 - ... denied the reports
 - ... denied the claims
 - ... denied the request
- Test set
 - ... denied the offer
 - ... denied the loan

$$P(\text{"offer"} \mid \text{denied the}) = 0$$



Language Modeling

Generalization and zeros



Zero probability bigrams

- Bigrams with zero probability
 - mean that we will assign 0 probability to the test set!
- And hence we cannot compute perplexity (can't divide by 0)!



Word Prediction

- So, we can predict future words in an utterance
- How?
 - Domain knowledge
 - Syntactic knowledge
 - Lexical knowledge
- We will use probabilities...



Word Prediction

- If you can predict the next word, you can predict the likelihood of sequences containing various alternative words.
- That will help us resolve POS, WSD, spelling correction, hand-writing recognition, speech recognition, augmentative communication



N-Grams

The big red **dog**

- Unigrams: $P(\text{dog})$
- Bigrams: $P(\text{dog} | \text{red})$
- Trigrams: $P(\text{dog} | \text{big red})$
- Four-grams: $P(\text{dog} | \text{the big red})$

In general, we'll be dealing with
 $P(\text{Word} | \text{Some fixed prefix})$



Using N-Grams

- $P(\text{I want to eat Chinese food}) = P(\text{I} | \text{start})P(\text{want} | \text{I})P(\text{to} | \text{I want}) \dots P(\text{food} | \text{I want to eat Chinese})$
- Markov assumptions
- Bigrams $(\text{I} | \text{start})P(\text{want} | \text{I})P(\text{to} | \text{want}) \dots P(\text{food} | \text{Chinese})$
- Trigrams
- $P(\text{I} | \text{start})P(\text{want} | \text{I})P(\text{to} | \text{I want}) \dots P(\text{food} | \text{eat Chinese})$



Raw bigram counts

- Out of 9222 sentences

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0



Raw bigram probabilities

- Normalize by unigrams:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Result:

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0



An Aside on Logs

- You don't really do all those multiplies. The numbers are too small and lead to underflows
- Convert the probabilities to logs and then do additions.



Generation

- Choose N-Grams with non-0 probabilities and string them together to get a feeling for accuracy of the N-gram model



Shakespeare

- **Unigrams:**

- Every enter now severally so, let
- Hill he late speaks; or! A more to leg less first you enter

- **Bigrams:**

- What means, sir. I confess she? Then all sorts, he is trim, captain.
- Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry.



Shakespeare

- Trigrams
 - Sweet prince, Falstaff shall die.
 - This shall forbid it should be branded, if renown made it empty
- Quadigrams
 - What! I will go seek the traitor Gloucester
 - Will you not tell me who I am?



Observations

- A small number of events occur with high frequency
 - You can collect reliable statistics on these events with relatively small samples
- A large number of events occur with small frequency
 - You might have to wait a long time to gather statistics on the low frequency events



Observations

- Some zeroes are really zeroes
 - Meaning that they represent events that can't or shouldn't occur
- On the other hand, some zeroes aren't really zeroes
 - They represent low frequency events that simply didn't occur in the corpus

Language Modeling

Smoothing: Add-one (Laplace) smoothing



The intuition of smoothing (from Dan Klein)

- When we have sparse statistics:

$P(w \mid \text{denied the})$

3 allegations

2 reports

1 claims

1 request

7 total

- Steal probability mass to generalize better

$P(w \mid \text{denied the})$

2.5 allegations

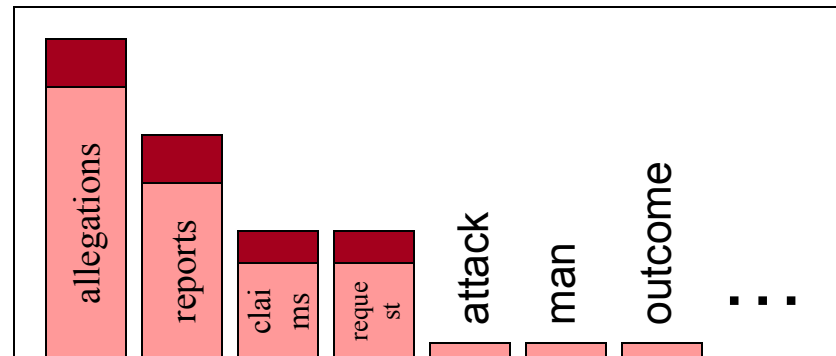
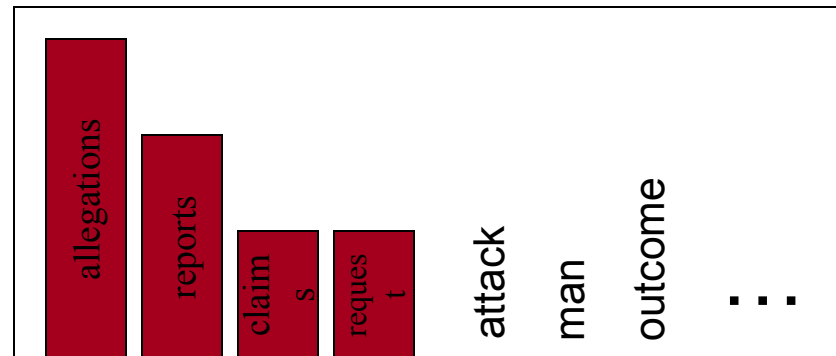
1.5 reports

0.5 claims

0.5 request

2 other

7 total





Add-one estimation

- Also called Laplace smoothing
- Pretend we saw each word one more time than we did
- Just add one to all the counts!

$$P_{MLE}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

- MLE estimate:

$$P_{Add-1}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + V}$$

- Add-1 estimate:



Maximum Likelihood Estimates

- The maximum likelihood estimate
 - of some parameter of a model M from a training set T
 - maximizes the likelihood of the training set T given the model M
- Suppose the word “bagel” occurs 400 times in a corpus of a million words
- What is the probability that a random word from some other text will be “bagel”?
- MLE estimate is $400/1,000,000 = .0004$
- This may be a bad estimate for some other corpus
 - But it is the **estimate** that makes it **most likely** that “bagel” will occur 400 times in a million word corpus.



Berkeley Restaurant Corpus: Laplace smoothed bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1



Important

Laplace-smoothed bigrams

$$P^*(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + V}$$

V= Vocabulary Size

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058



Reconstituted counts

$$c^*(w_{n-1}w_n) = \frac{[C(w_{n-1}w_n) + 1] \times C(w_{n-1})}{C(w_{n-1}) + V}$$

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16



Compare with raw bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16



Add-one Smoothing (Laplace's law)

- Pretend we have seen every n-gram at least once
- Intuitively:
 - $\text{new_count}(\text{n-gram}) = \text{old_count}(\text{n-gram}) + 1$
- The idea is to give a little bit of the probability space to unseen events
- $P(w_i | w_{i-1}) = \frac{(\text{count}(w_{i-1} w_i) + 1)}{(\text{count}(w_{i-1}) + V)}$
- Later slides: $\text{count}(w_{i-1})$ – referred to as N

unsmoothed bigram counts: $\uparrow$ 2nd word

	<i>I</i>	<i>want</i>	<i>to</i>	<i>eat</i>	<i>Chinese</i>	<i>food</i>	<i>lunch</i>	...	Total (N)
1 st word {	<i>I</i>	8	1087	0	13	0	0	0	3437
	<i>want</i>	3	0	786	0	6	8	6	1215
	<i>to</i>	3	0	10	860	3	0	12	3256
	<i>eat</i>	0	0	2	0	19	2	52	938
	<i>Chinese</i>	2	0	0	0	0	120	1	213
	<i>food</i>	19	0	17	0	0	0	0	1506
	<i>lunch</i>	4	0	0	0	0	1	0	459
	...								

unsmoothed normalized bigram probabilities:

[illegible]



Add-one: Example (V=1616)

add-one smoothed bigram counts:

	<i>I</i>	<i>want</i>	<i>to</i>	<i>eat</i>	<i>Chinese</i>	<i>food</i>	<i>lunch</i>	...	Total (N+V)
<i>I</i>	8 9 1087 1088	1	14	1	1	1	1		3437 5053
<i>want</i>	3 4	1	787	1	7	9	7		2831
<i>to</i>	4	1	11	861	4	1	13		4872
<i>eat</i>	1	1	23	1	20	3	53		2554
<i>Chinese</i>	3	1	1	1	1	121	2		1829
<i>food</i>	20	1	18	1	1	1	1		3122
<i>lunch</i>	5	1	1	1	1	2	1		2075

add-one normalized bigram probabilities:

	<i>I</i>	<i>want</i>	<i>to</i>	<i>eat</i>	<i>Chinese</i>	<i>food</i>	<i>lunch</i>	...	Total
<i>I</i>	.0018 (9/5053)	.22	.0002	.0028 (14/5053)	.0002	.0002	.0002		1
<i>want</i>	.0014	.00035	.28	.00035	.0025	.0032	.0025		1
<i>to</i>	.00082	.00021	.0023	.18	.00082	.00021	.0027		1
<i>eat</i>	.00039	.00039	.0012	.00039	.0078	.0012	.021		1
<i>Chinese</i>	.0016	.00055	.00055	.00055	.00055	.066	.0011		1
<i>food</i>	.0064	.00032	.0058	.00032	.00032	.00032	.00032		1
<i>lunch</i>	.0024	.00048	.00048	.00048	.00048	.0022	.00048		1



Problem add-one smoothing ($V=1616$)

- bigrams starting with *Chinese* are boosted by a factor of 8 ! (1829 / 213)

unsmoothed bigram counts:

	<i>I</i>	<i>want</i>	<i>to</i>	<i>eat</i>	<i>Chinese</i>	<i>food</i>	<i>lunch</i>	...	Total (N)
<i>I</i>	8	1087	0	13	0	0	0		3437
<i>want</i>	3	0	786	0	6	8	6		1215
<i>to</i>	3	0	10	860	3	0	12		3256
<i>eat</i>	0	0	2	0	19	2	52		938
<i>Chinese</i>	2	0	0	0	0	120	1		<u>213</u>
<i>food</i>	19	0	17	0	0	0	0		1506
<i>lunch</i>	4	0	0	0	0	1	0		459

add-one smoothed bigram counts:

	<i>I</i>	<i>want</i>	<i>to</i>	<i>eat</i>	<i>Chinese</i>	<i>food</i>	<i>lunch</i>	...	Total (N+V)
<i>I</i>	9	1088	1	14	1	1	1		5053
<i>want</i>	4	1	787	1	7	9	7		2831
<i>to</i>	4	1	11	861	4	1	13		4872
<i>eat</i>	1	1	23	1	20	3	53		2554
<i>Chinese</i>	3	1	1	1	1	121	2		<u>1829</u>
<i>food</i>	20	1	18	1	1	1	1		3122
<i>lunch</i>	5	1	1	1	1	2	1		2075



Exercise

- **Corpus (3 lines)**
- `<s1> I like apples </s>`
- `<s2> I like bananas </s>`
- `<s3> I hate apples </s>`
- Calculate the Laplace-smoothed probability of the bigram "**like apples**": (ignore start symbol in vocabulary count) (Raw count = 6)
- Question 2
- **Corpus (4 lines)**



- $\langle s1 \rangle$ cat sat on mat $\langle /s \rangle$
- $\langle s2 \rangle$ cat sat on log $\langle /s \rangle$
- $\langle s3 \rangle$ dog sat on mat $\langle /s \rangle$
- $\langle s4 \rangle$ cat slept on mat $\langle /s \rangle$
- Vocabulary
- The unique words are:
- $\{\text{cat, dog, sat, slept, on, mat, log, } \langle /s \rangle\} = 8$
- Task
- Calculate the Laplace-smoothed probability for an unseen bigram "dog slept":



Exercise 3

- **Corpus (3 lines)**
- the dog chased the cat
- the cat chased the mouse
- the dog chased the mouse
- **Vocabulary (\$V\$)**
- The unique words in this corpus are:
- **{the, dog, chased, cat, mouse} = 5**
- **Task**
- Calculate the Laplace-smoothed probability of the bigram "**chased the**":



Exercise 4

- **Corpus (4 lines)**
- **blue sky above**
- **blue ocean below**
- **clear sky above**
- **deep ocean below**
- **Vocabulary (\$V\$)**
- **The unique words in this corpus are:**
- **{blue, sky, above, ocean, below, clear, deep} = 7**
- **Task**
- **Calculate the Laplace-smoothed probability of the bigram "blue sky" vs. "blue below":**



Problem with add-one smoothing

- every previously unseen n-gram is given a low probability
- but there are so many of them that too much probability mass is given to unseen events
- adding 1 to frequent bigram, does not change much
- but adding 1 to low bigrams (including unseen ones) boosts them too much !
- In NLP applications that are very sparse, Laplace's Law actually gives far too much of the probability space to unseen events.



Add-1 estimation is a blunt instrument

- So add-1 isn't used for N-grams:
 - We'll see better methods
- But add-1 is used to smooth other NLP models
 - For text classification
 - In domains where the number of zeros isn't so huge.



Witten-Bell smoothing

- intuition:
 - An unseen n-gram is one that just did not occur yet
 - When it does happen, it will be its first occurrence
 - So give to unseen n-grams the probability of seeing a new n-gram



Witten-Bell: the equations

- Total probability mass assigned to zero-frequency unigrams (T #observed types; N #word instances/tokens):

$$\sum_{i:c_i=0} p_i^* = \frac{T}{N + T}$$

- So each zero N-gram gets the probability:

$$Z = \sum_{i:c_i=0} 1$$

$$p_i^* = \frac{T}{Z(N + T)}$$



Witten-Bell: why ‘discounting’

- Now of course we have to take away something (‘discount’) from the probability of the events seen more than once:

If $c_i > 0$,

$$p_i^* = \frac{c_i}{N + T}$$



Witten-Bell for bigrams

- We 'relativize' the types to the previous word:

$$\sum_{i:c(w_x w_i)=0} p^*(w_i \mid w_x) = \frac{T(w_x)}{N(w_x) + T(w_x)}$$

- this probability mass, must be distributed in equal parts over all unseen bigrams
 - $Z(w_1)$: number of unseen n-grams starting with w_1

for each unseen event

$$P(w_2|w_1) = \frac{1}{Z(w_1)} \times \frac{T(w_1)}{N(w_1) + T(w_1)}$$



Small example

	a	b	c	d		Total = N(w1) nb seen tokens	T(w1) nb seen types	Z(w1) nb. unseen types
a	10	10	10	0		30	3	1
b	0	0	30	0		30	1	3
c	0	0	300	0		300	1	3
d								
...								

- all unseen bigrams starting with *a* will share a probability mass of

$$\frac{T(a)}{N(a) + T(a)} = \frac{3}{30 + 3} = 0.091$$

- each unseen bigram starting with *a* will have an equal part of this

$$P(d|a) = \frac{1}{Z(a)} \times \frac{T(a)}{N(a) + T(a)} = \frac{1}{1} \times 0.091 = 0.091$$



Small example (con't)

	a	b	c	d	...	Total = N(w1) nb seen tokens	T(w1) nb seen types	Z(w1) nb. unseen types
a	10	10	10	0		30	3	1
b	0	0	30	0		30	1	3
c	0	0	300	0		300	1	3
d								
...								

- all unseen bigrams starting with *b* will share a probability mass of

$$\frac{T(b)}{N(b) + T(b)} = \frac{1}{30 + 1} = 0.032$$

- each unseen bigrams starting with *b* will have an equal part of this

$$P(ab) = \frac{1}{Z(b)} \times \frac{T(b)}{N(b) + T(b)} = \frac{1}{3} \times 0.032 = 0.011$$

$$P(bb) = \frac{1}{Z(b)} \times \frac{T(b)}{N(b) + T(b)} = \frac{1}{3} \times 0.032 = 0.011$$

$$P(db) = \frac{1}{Z(b)} \times \frac{T(b)}{N(b) + T(b)} = \frac{1}{3} \times 0.032 = 0.011$$



Small example (con't)

	a	b	c	d	...	Total = N(w1) nb seen tokens	T(w1) nb seen types	Z(w1) nb. unseen types
a	10	10	10	0		30	3	1
b	0	0	30	0		30	1	3
c	0	0	300	0		300	1	3
d								
...								

- all unseen bigrams starting with c will share a probability mass of

$$\frac{T(c)}{N(c) + T(c)} = \frac{1}{300 + 1} = 0.0033$$
- each unseen bigrams starting with c will have an equal part of this

$$P(a|c) = \frac{1}{Z(c)} \times \frac{T(c)}{N(c) + T(c)} = \frac{1}{3} \times 0.0033 = 0.0011$$

$$P(b|c) = \frac{1}{Z(c)} \times \frac{T(c)}{N(c) + T(c)} = \frac{1}{3} \times 0.0033 = 0.0011$$

$$P(d|c) = \frac{1}{Z(c)} \times \frac{T(c)}{N(c) + T(c)} = \frac{1}{3} \times 0.0033 = 0.0011$$



Back to Counts

- Unseen bigrams:
 - To get from probabilities back to the counts, we know that:

$$P(w_2|w_1) = \frac{C(w_2|w_1)}{N(w_1)} \quad // N(w_1) = \text{nb of bigrams starting with } w_1$$

//C(w2 | w1) here means Count(w1 w2)

- so we get:

$$\begin{aligned} C(w_2|w_1) &= P(w_2|w_1) \times N(w_1) \\ &= \frac{1}{Z(w_1)} \times \frac{T(w_1)}{N(w_1) + T(w_1)} \times N(w_1) \\ &= \frac{T(w_1)}{Z(w_1)} \times \frac{N(w_1)}{N(w_1) + T(w_1)} \end{aligned}$$



The restaurant example

- The original counts were:

	<i>I</i>	<i>want</i>	<i>to</i>	<i>eat</i>	<i>Chinese</i>	<i>food</i>	<i>lunch</i>	...	<i>N(w)</i> <i>seen bigram</i> <i>tokens</i>	<i>T(w)</i> <i>seen bigram</i> <i>types</i>	<i>Z(w)</i> <i>unseen</i> <i>bigram types</i>
<i>I</i>	8	1087	0	13	0	0	0		3437	95	1521
<i>want</i>	3	0	786	0	6	8	6		1215	76	1540
<i>to</i>	3	0	10	860	3	0	12		3256	130	1486
<i>eat</i>	0	0	2	0	19	2	52		938	124	1492
<i>Chinese</i>	2	0	0	0	0	120	1		213	20	1592
<i>food</i>	19	0	17	0	0	0	0		1506	82	534
<i>lunch</i>	4	0	0	0	0	1	0		459	45	1571

- $T(w)$ = number of different seen bigrams types starting with w
- we have a vocabulary of 1616 words, so we can compute
- $Z(w)$ = number of unseen bigrams types starting with w

$$Z(w) = 1616 - T(w)$$
- $N(w)$ = number of bigrams tokens starting with w



Witten-Bell smoothed count

- the count of the unseen bigram “*I lunch*”

$$\frac{T(I)}{Z(I)} \times \frac{N(I)}{N(I) + T(I)} = \frac{95}{1521} \times \frac{3437}{3437 + 95} = 0.06$$

- the count of the seen bigram “*want to*”

$$\text{count}(\text{want to}) \times \frac{N(\text{want})}{N(\text{want}) + T(\text{want})} = 786 \times \frac{1215}{1215 + 76} = 739.73$$

Witten-Bell smoothed bigram counts:

	<i>I</i>	<i>want</i>	<i>to</i>	<i>eat</i>	<i>Chinese</i>	<i>food</i>	<i>lunch</i>	...	Total
<i>I</i>	7.78	1057.76	.061	12.65	.06	.06	.06		3437
<i>want</i>	2.82	.05	739.73	.05	5.65	7.53	5.65		1215
<i>to</i>	2.88	.08	9.62	826.98	2.88	.08	12.50		3256
<i>eat</i>	.07	.07	19.43	.07	16.78	1.77	45.93		938
<i>Chinese</i>	1.74	.01	.01	.01	.01	109.70	.91		213
<i>food</i>	18.02	.05	16.12	.05	.05	.05	.05		1506
<i>lunch</i>	3.64	.03	.03	.03	.03	0.91	.03		459



Witten-Bell smoothed probabilities

Witten-Bell normalized bigram probabilities:

Error: one too many 0s

	<i>I</i>	<i>want</i>	<i>to</i>	<i>eat</i>	<i>Chinese</i>	<i>food</i>	<i>lunch</i>	...	Total
<i>I</i>	.0022 (7.78/3437)	.3078	.000002	.0037	.000002	.000002	.000002		1
<i>want</i>	.00230	.00004	.6088	.00004	.0047	.0062	.0047		1
<i>to</i>	.00009	.00003	.0030	.2540	.00009	.00003	.0038		1
<i>eat</i>	.00008	.00008	.0021	.00008	.0179	.0019	.0490		1
<i>Chinese</i>	.00812	.00005	.00005	.00005	.00005	.5150	.0042		1
<i>food</i>	.0120	.00004	.0107	.00004	.00004	.00004	.00004		1
<i>lunch</i>	.0079	.00006	.00006	.00006	.00006	.0020	.00006		1



Simple Linear Interpolation

- Solve the sparseness in a trigram model by mixing with bigram and unigram models
- Also called:
 - linear interpolation,
 - finite mixture models
 - deleted interpolation
- Combine linearly

$$P_{li}(w_n | w_{n-2}, w_{n-1}) = \lambda_1 P(w_n) + \lambda_2 P(w_n | w_{n-1}) + \lambda_3 P(w_n | w_{n-2}, w_{n-1})$$

- where $0 \leq \lambda_i \leq 1$ and $\sum_i \lambda_i = 1$